

IP PORT SCANNER

A project report submitted in partial fulfillment of the
requirements for the digital Assignment of

Computer Network

by

DARSH SONI & PRITISHA MISHRA

(24BRS1207, 24BRS1204)



VIT[®]
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

Vellore Institute of
Technology, Chennai,
VandalurKelambakkam
Road,
Chennai - 600127

DECLARATION

We hereby declare that the report titled **IP Port Scanner** submitted by us to theVellore Institute of Technology, Chennai in partial fulfillment of the requirements for the award of Digital Assignment of **Computer Network** is a bona-fide record of the work carried out by us under the supervision of **Dr. Neelanarayan V.**

Sign:

Name & Reg. No.: Darsh Soni (24BRS1207) , Pritisha Mishra (24BRS1204)

Date: 07th November 2025

.

Problem Statement

In modern computer networks, identifying open and closed ports on a device is essential for diagnosing connectivity issues, ensuring proper configuration, and maintaining security. However, most existing tools are complex or require advanced knowledge. Therefore, there is a need for a simple, python-based port scanning system that can detect open ports across different networks. This project aims to develop an IP Port Scanner capable of identifying port availability on remote devices using socket programming.

SOLUTION

The solution to the identified problem is to develop a simple and efficient IP Port Scanner using the python programming language and socket programming techniques. The system is designed to scan a specified range of ports on a target device and determine which ports are open, closed, or inaccessible. This is achieved by attempting to create a TCP socket connection with each port. If the connection is successfully established, the program identifies the port as open; otherwise, it marks it as closed.

The scanner uses fundamental networking concepts such as IP addressing, socket creation, connection handling, and error detection. By integrating commands like *ipconfig* and using the device's network interface information, the program ensures accurate communication with the target system. One of the highlights of the solution is its ability to operate across different networks—for example, two laptops connected through different mobile hotspots can still detect each other's port status.

This capability demonstrates real-world networking behavior and showcases how devices interact across networks using ports as communication endpoints. The solution is lightweight, fast, and user-friendly, making it ideal for learning environments and basic network diagnostics. It provides transparency into how port scanning works internally, unlike complex external tools, and reinforces classroom concepts through practical implementation.

REQUIREMENT

1. Hardware Requirements

- **Two Laptops/Computers** for testing port scanning across different networks
- **Mobile Hotspot or Wi-Fi Network** (e.g., hotspot from two different devices)
- **Basic Networking Hardware** (network adapter, router, etc.)
- **Minimum 2 GB RAM**
- **Processor:** Dual-core or higher

2. Software Requirements

- **Operating System:** Windows / Linux
- **C Compiler:** GCC (Linux) or MinGW (Windows)
- **IDE/Editor:** Code::Blocks, VS Code, or any **Socket Programming Libraries:** Built-in system libraries (winsock2.h for Windows or sys/socket.h for Linux)
- **Command Line Terminal:** CMD, PowerShell, or Linux terminal for execution
- **IP Configuration Tools:** **ipconfig** (Windows) or **ifconfig** (Linux)

3. Functional Requirements

- Ability to input a target **IP address**
- Ability to specify a **range of ports** for scanning

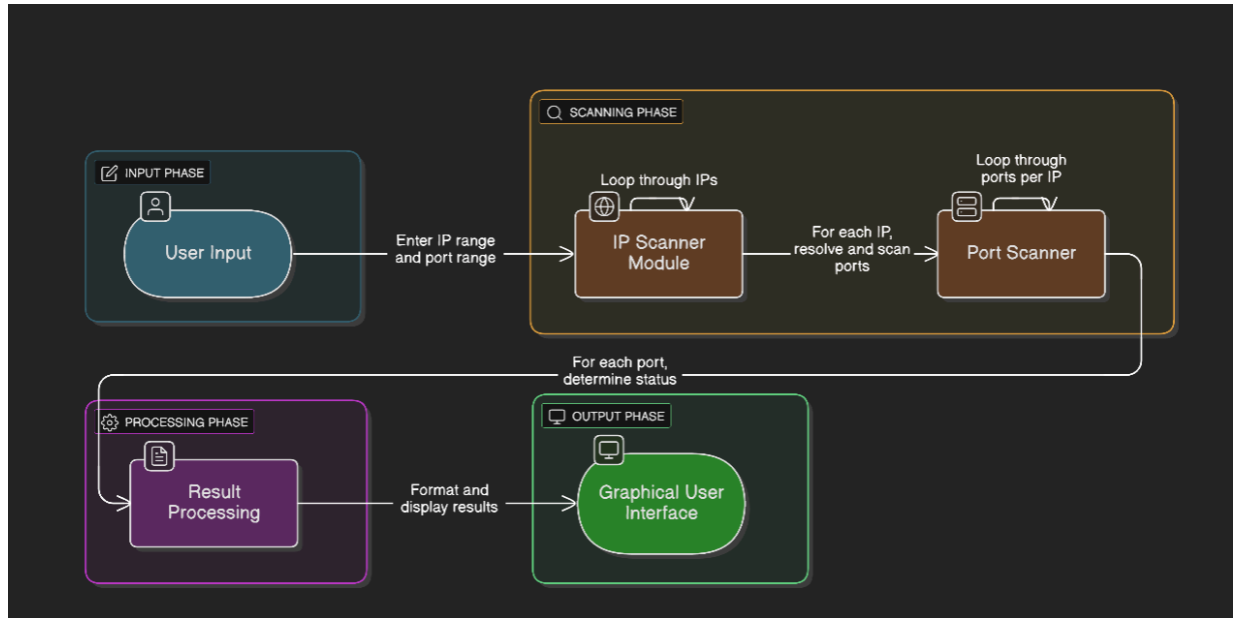
- System must check connection status for each port
- Display results showing **open** and **closed** ports
- Handle network errors or unreachable devices gracefully

4. Non-Functional Requirements

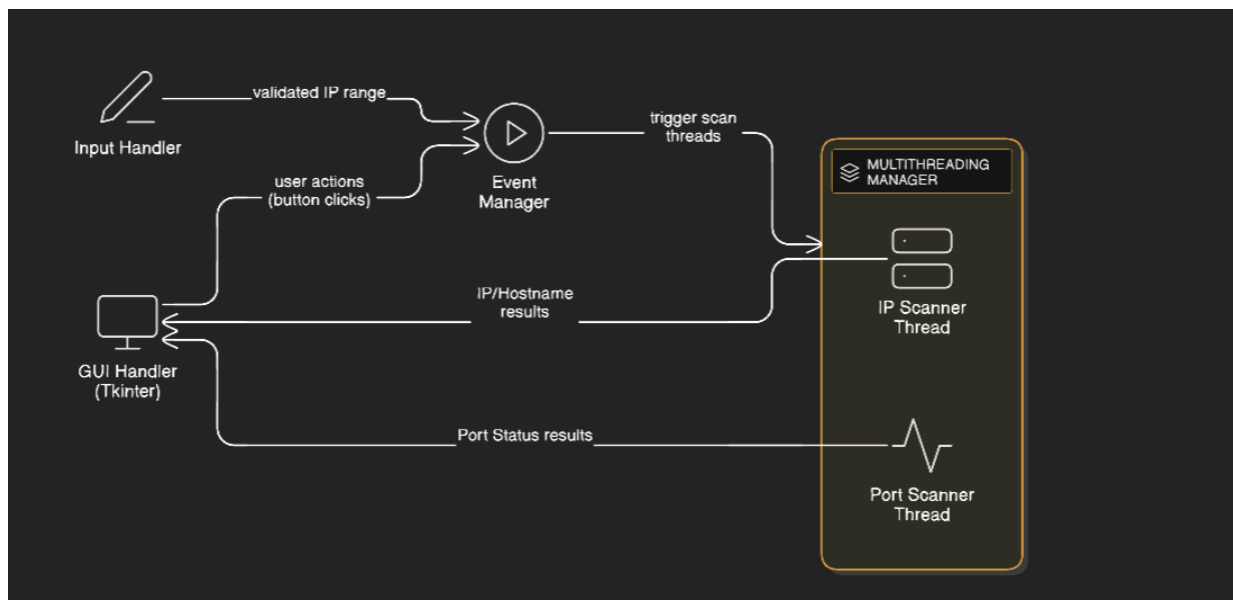
- Program should execute efficiently and scan ports quickly
- User-friendly command-line interface
- Should work across different networks (e.g., different hotspots)
- Must ensure stable and secure handling of socket connections

DESIGN

HIGH LEVEL DIARAM

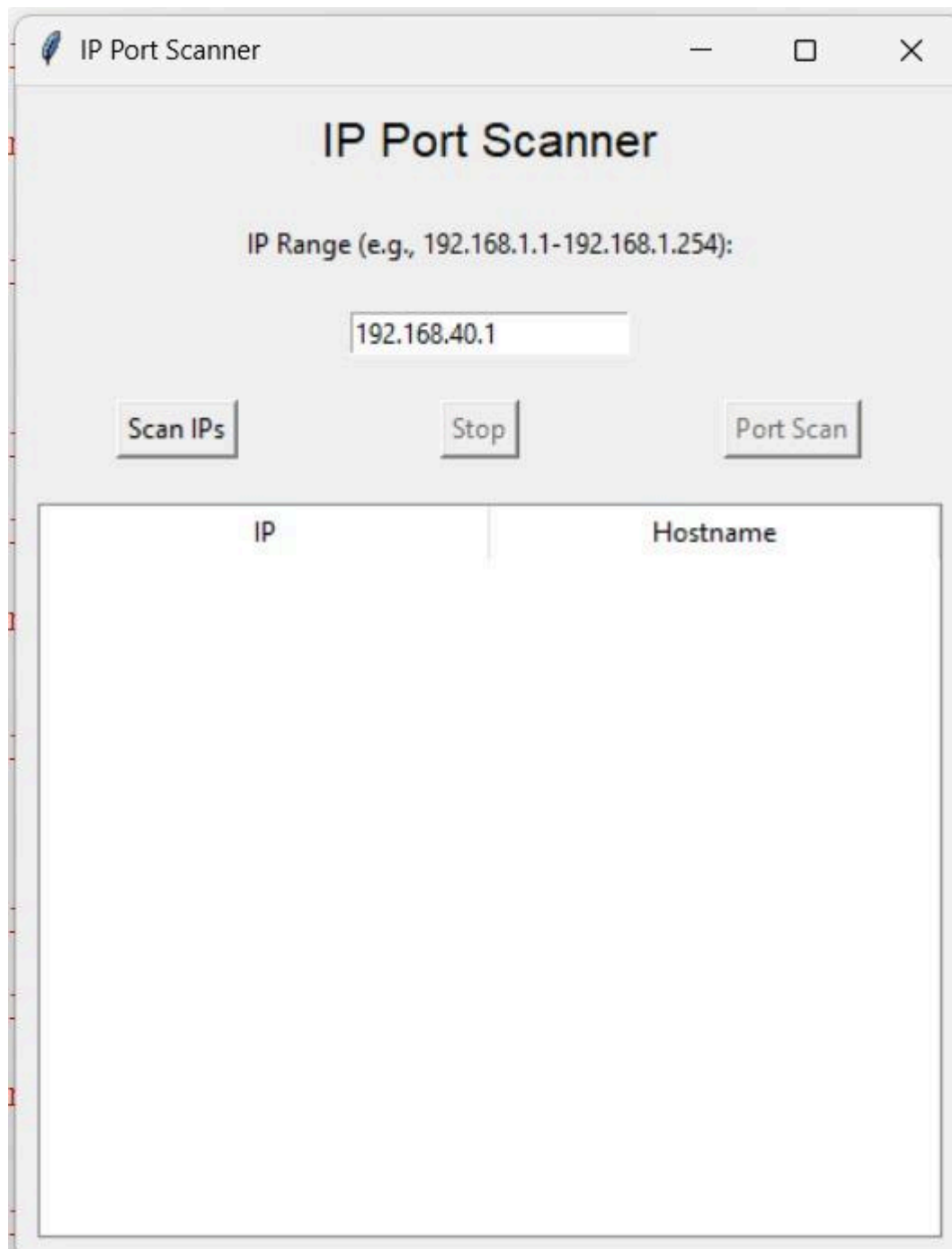


LOW LEVEL DIAGRAM



Component	Description	Connections (Data/Control Flow)
A. Input Handler	Takes input, parses, and validates the IP range.	Sends validated IP range to the Event Manager (when a scan starts).
E. GUI Handler (Tkinter)	Handles user interface (buttons, text box, TreeView).	Receives status updates and scan results from the IP Scanner and Port Scanner threads. Sends user actions (button clicks) to the Event Manager .
F. Event Manager	Receives button presses and initiates scans.	Triggers the Multithreading Manager to start a new IP Scanner Thread or Port Scanner Thread .
D. Multithreading Manager	Creates and manages separate threads for scanning.	Starts B. IP Scanner Thread and C. Port Scanner Thread . Handles thread stopping.
B. IP Scanner Thread	Iterates IP addresses, uses <code>socket.gethostbyaddr()</code> .	Sends IP/Hostname results back to the GUI Handler .
C. Port Scanner Thread	Scans ports for a selected IP using <code>connect_ex()</code> .	Sends Port Status (Open/Closed) results back to the GUI Handler .

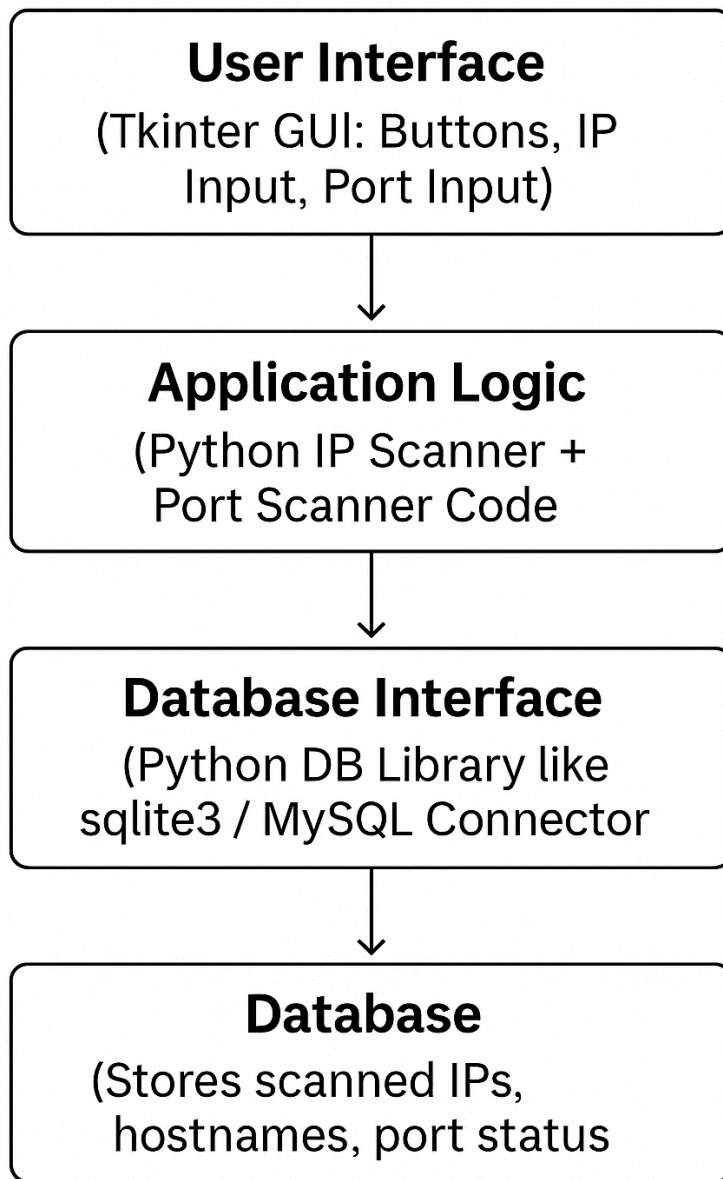
USER INTERFACE



The image shows a graphical user interface for an "IP Port Scanner". The window has a title bar with the text "IP Port Scanner" and standard minimize, maximize, and close buttons. The main content area has a title "IP Port Scanner" and a label "IP Range (e.g., 192.168.1.1-192.168.1.254):". Below this label is a text input field containing the IP address "192.168.40.1". There are three buttons: "Scan IPs", "Stop", and "Port Scan". Below the buttons is a table with two columns: "IP" and "Hostname". The table is currently empty.

IP	Hostname
----	----------

DATABASE INTERFACE



SAMPLE CODE

```
import tkinter as tk
import socket
import threading
import tkinter.ttk as ttk
from tkinter import messagebox

# Main Application Class
class IPScannerApp:
    def __init__(self, root):
        self.root = root
        self.root.title("IP Port Scanner")

        # Title
        software_label = tk.Label(root, text="IP Port Scanner", font=("Helvetica",
16))
        software_label.grid(row=0, column=0, columnspan=3, padx=10, pady=10)

        # Scan state
        self.scan_in_progress = False
        self.scan_thread = None

        # IP Range Input
        self.ip_range_label = tk.Label(root, text="IP Range (e.g.,
192.168.1.1-192.168.1.254):")
        self.ip_range_label.grid(row=1, column=0, columnspan=3, padx=10,
pady=10)
```

```

self.ip_range_entry = tk.Entry(root)

self.ip_range_entry.grid(row=2, column=0, columnspan=3, padx=10,
pady=10)


# Buttons

self.scan_button = tk.Button(root, text="Scan IPs",
command=self.start_ip_scan)

self.scan_button.grid(row=3, column=0, padx=10, pady=10)


self.stop_button = tk.Button(root, text="Stop", command=self.stop_scan,
state=tk.DISABLED)

self.stop_button.grid(row=3, column=1, padx=10, pady=10)


self.port_scan_button = tk.Button(root, text="Port Scan",
command=self.start_port_scan, state=tk.DISABLED)

self.port_scan_button.grid(row=3, column=2, padx=10, pady=10)


# Tree to show scanned IPs

self.tree = ttk.Treeview(root, columns=("IP", "Hostname"),
show="headings", height=15)

self.tree.heading("IP", text="IP")

self.tree.heading("Hostname", text="Hostname")

self.tree.grid(row=4, column=0, columnspan=3, padx=10, pady=10)


# IP Scan

def start_ip_scan(self):

    if self.scan_in_progress:

```

```
messagebox.showinfo("Info", "Scan already in progress")  
return
```

```
ip_range = self.ip_range_entry.get()
```

```
try:
```

```
    start_ip, end_ip = ip_range.split('-')
```

```
    base = ".".join(start_ip.split('.')[:-1])
```

```
    start_int = int(start_ip.split('.')[-1])
```

```
    end_int = int(end_ip.split('.')[-1])
```

```
except:
```

```
    messagebox.showerror("Error", "Invalid IP range format")
```

```
    return
```

```
self.scan_in_progress = True
```

```
self.scan_button.config(state=tk.DISABLED)
```

```
self.stop_button.config(state=tk.NORMAL)
```

```
self.tree.delete(*self.tree.get_children())
```

```
def scan_ip(ip):
```

```
    try:
```

```
        hostname = socket.gethostbyaddr(ip)[0]
```

```
    except:
```

```
        hostname = "N/A"
```

```
    self.tree.insert("", "end", values=(ip, hostname))
```

```

def scan_range():
    for i in range(start_int, end_int + 1):
        if not self.scan_in_progress:
            break
        ip = f"{base}.{i}"
        scan_ip(ip)

    self.scan_in_progress = False
    self.scan_button.config(state=tk.NORMAL)
    self.stop_button.config(state=tk.DISABLED)
    self.port_scan_button.config(state=tk.NORMAL)

    self.scan_thread = threading.Thread(target=scan_range)
    self.scan_thread.start()

# Stop IP Scan
def stop_scan(self):
    self.scan_in_progress = False

# Open port-scan window for selected IP
def start_port_scan(self):
    selected = self.tree.selection()
    if not selected:
        messagebox.showinfo("Info", "Select an IP first")
    return

```

```
ip = self.tree.item(selected[0], "values")[0]
```

```
port_window = tk.Toplevel(self.root)
```

```
PortScanner(port_window, ip)
```

```
# Port Scanner Class
```

```
class PortScanner:
```

```
    def __init__(self, root, ip):
```

```
        self.root = root
```

```
        self.ip = ip
```

```
        self.root.title(f"Port Scanner - {ip}")
```

```
        self.scan_in_progress = False
```

```
        self.scan_thread = None
```

```
    # Port range input
```

```
        tk.Label(root, text="Port Range (e.g., 1-1024):").grid(row=0, column=0,  
columnspan=2, padx=10, pady=10)
```

```
        self.port_range_entry = tk.Entry(root)
```

```
        self.port_range_entry.grid(row=1, column=0, columnspan=2, padx=10,  
pady=10)
```

```
    # Buttons
```

```
        tk.Button(root, text="Scan Ports",  
command=self.start_port_scan).grid(row=2, column=0, padx=10, pady=10)
```

```

self.stop_button = tk.Button(root, text="Stop",
command=self.stop_port_scan, state=tk.DISABLED)

self.stop_button.grid(row=2, column=1, padx=10, pady=10)

# Tree for port results

self.tree = ttk.Treeview(root, columns=("Port", "Status"), show="headings",
height=15)

self.tree.heading("Port", text="Port")
self.tree.heading("Status", text="Status")

self.tree.grid(row=3, column=0, columnspan=2, padx=10, pady=10)

# Start port scan

def start_port_scan(self):
    if self.scan_in_progress:
        messagebox.showinfo("Info", "Scan already in progress")
        return

    try:
        start_port, end_port = map(int, self.port_range_entry.get().split('-'))
    except:
        messagebox.showerror("Error", "Invalid port range")
        return

    self.scan_in_progress = True
    self.stop_button.config(state=tk.NORMAL)
    self.tree.delete(*self.tree.get_children())

```

```
def scan_port(port):  
    sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)  
    sock.settimeout(0.5)  
    result = sock.connect_ex((self.ip, port))  
    status = "Open" if result == 0 else "Closed"  
    sock.close()  
    self.tree.insert("", "end", values=(port, status))
```

```
def scan_range():  
    for port in range(start_port, end_port + 1):  
        if not self.scan_in_progress:  
            break  
        scan_port(port)  
  
    self.scan_in_progress = False  
    self.stop_button.config(state=tk.DISABLED)  
  
self.scan_thread = threading.Thread(target=scan_range)  
self.scan_thread.start()
```

Stop port scan

```
def stop_port_scan(self):  
    self.scan_in_progress = False
```

Main execution

```
if __name__ == "__main__":  
    root = tk.Tk()  
    IPScannerApp(root)  
    root.mainloop()
```

SAMPLE UI

```
PS C:\Users\user> ipconfig

Windows IP Configuration

Ethernet adapter Ethernet:

    Media State . . . . . : Media disconnected
    Connection-specific DNS Suffix  . :

Wireless LAN adapter Local Area Connection* 1:

    Media State . . . . . : Media disconnected
    Connection-specific DNS Suffix  . :

Wireless LAN adapter Local Area Connection* 12:

    Media State . . . . . : Media disconnected
    Connection-specific DNS Suffix  . :

Ethernet adapter VMware Network Adapter VMnet1:

    Connection-specific DNS Suffix  . :
    Link-local IPv6 Address . . . . . : fe80::29db:e00a:2ff2:f599%10
    IPv4 Address. . . . . : 192.168.40.1
    Subnet Mask . . . . . : 255.255.255.0
    Default Gateway . . . . . :

Ethernet adapter VMware Network Adapter VMnet8:

    Connection-specific DNS Suffix  . :
    Link-local IPv6 Address . . . . . : fe80::5c75:b6a9:c2c7:92b%11
    IPv4 Address. . . . . : 192.168.17.1
    Subnet Mask . . . . . : 255.255.255.0
    Default Gateway . . . . . :

Wireless LAN adapter Wi-Fi:
```

Fig: Terminal View

IP Port Scanner

IP Range (e.g., 192.168.1.1-192.168.1.254):

92.168.40.1-192.168.40.:

Scan IPs

Stop

Port Scan

IP	Hostname
192.168.1.1	N/A
192.168.1.2	N/A

Port Range (e.g., 1-1024):

1-100

Scan Ports Stop

Port	Status
3	Closed
4	Closed
5	Closed
6	Closed
7	Closed
8	Closed
9	Closed
10	Closed
11	Closed
12	Closed
13	Closed
14	Closed
15	Closed
16	Closed
17	Closed

Fig: Port Range View

Port Range (e.g., 1-1024):

1-100

Scan Ports

Stop

Port	Status
22	Closed
23	Closed
24	Closed
25	Open
26	Closed
27	Closed
28	Closed
29	Closed
30	Closed
31	Closed
32	Closed
33	Closed
34	Closed
35	Closed
36	Closed

CONCLUSION

The IP Port Scanner developed using socket programming successfully demonstrates how network devices communicate through ports and how their availability can be detected remotely. By scanning a range of ports and identifying which ones are open or closed, the project provides valuable insight into real-world networking concepts such as IP addressing, TCP connections, and network accessibility. Testing the system across different networks—such as separate mobile hotspots—proves its reliability and practical effectiveness in remote scanning scenarios.

This project not only meets its objective of creating a simple, efficient, and educational port scanning tool but also strengthens understanding of core networking principles taught in class. It offers a transparent view of how port scanners operate internally, unlike complex external tools. Overall, the project enhances both theoretical and practical knowledge, making it a meaningful learning experience in network programming.

FUTURE ENHANCEMENT

Graphical User Interface (GUI):

The port scanner can be upgraded from a command-line tool to a user-friendly graphical interface, making it easier for non-technical users to operate.

1. UDP Port Scanning:

The current implementation focuses mainly on TCP ports. Adding support for **UDP scanning** would expand the tool's capabilities and provide more comprehensive results.

2. Multi-threaded Scanning:

Implementing multi-threading can significantly increase scanning speed by checking multiple ports simultaneously instead of sequentially.

3. Service Detection:

The scanner can be improved to identify which **service or application** is running on each open port (e.g., HTTP, FTP, SSH).

4. Logging and Reporting:

Adding features to save scan results as **PDF, text, or log files** would make the tool more suitable for documentation and analysis.

5. Firewall and Security Analysis:

The system can be enhanced to detect filtered ports and firewall restrictions to provide more detailed network security insights.

6. Cross-Platform Packaging:

Creating installable versions for Windows, Linux, and macOS would make the scanner portable and convenient to use across multiple systems.